

Министерство науки и высшего образования Российской Федерации

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»

Институт информатики и кибернетики

Кафедра технической кибернетики

Отчет по лабораторной работе №5

Дисциплина: «ООП»

Тема «Методы класса Object»

Выполнил: Чечеткин Д.А.

Группа: 6201-120303D

Самара, 2025

Задание 1

В классе **FunctionPoint** переопределены методы:

```
@Override
public String toString() { return "(" + x + "; " + y + ")"; }
```

Возвращает текстовое описание точки

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    FunctionPoint that = (FunctionPoint) o;
    return Math.abs(this.x - that.x) < EPS &&
           Math.abs(this.y - that.y) < EPS;
}
```

Сравнивает две точки, возвращая true при точном совпадении

```
@Override
public int hashCode() {
    long xBits = Double.doubleToLongBits(x);
    long yBits = Double.doubleToLongBits(y);

    int xLow = (int)(xBits & 0xFFFFFFFFL);
    int xHigh = (int)(xBits >>> 32);

    int yLow = (int)(yBits & 0xFFFFFFFFL);
    int yHigh = (int)(yBits >>> 32);

    return xLow ^ xHigh ^ yLow ^ yHigh;
}
```

Возвращает значение хэш-кода для объекта точки

```
@Override
public FunctionPoint clone() {
    try {
        return (FunctionPoint) super.clone();
    } catch (CloneNotSupportedException e) {
        throw new RuntimeException("клонирование не сработало", e);
    }
}
```

Возвращает объект-копию для объекта точки

Задание 2

В классе **ArrayTabulatedFunction** переопределены методы:

```
@Override
public String toString() {
    StringBuilder sb = new StringBuilder();
    sb.append("{");
    for (int i = 0; i < pointsCount; i++) {
        if (i > 0) {
            sb.append(", ");
        }
        sb.append("(").append(points[i].getX()).append("; ").append(points[i].getY()).append(")");
    }
    sb.append("}");
    return sb.toString();
}
```

Возвращает описание табулированной функции

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;

    if (o instanceof ArrayTabulatedFunction) {
        ArrayTabulatedFunction other = (ArrayTabulatedFunction) o;
        if (this.pointsCount != other.pointsCount) return false;

        for (int i = 0; i < pointsCount; i++) {
            if (Math.abs(this.points[i].getX() - other.points[i].getX()) >= EPS ||
                Math.abs(this.points[i].getY() - other.points[i].getY()) >= EPS) {
                return false;
            }
        }
        return true;
    }
    if (o instanceof TabulatedFunction) {
        TabulatedFunction other = (TabulatedFunction) o;
        if (this.getPointsCount() != other.getPointsCount()) return false;

        for (int i = 0; i < pointsCount; i++) {
            FunctionPoint thisPoint = this.getPoint(i);
            FunctionPoint otherPoint = other.getPoint(i);

            if (Math.abs(thisPoint.getX() - otherPoint.getX()) >= EPS ||
                Math.abs(thisPoint.getY() - otherPoint.getY()) >= EPS) {
                return false;
            }
        }
        return true;
    }
    return false;
}
```

Сравнивает две табулированные функции

```

@Override
public int hashCode() {
    int hash = pointsCount;

    for (int i = 0; i < pointsCount; i++) {
        int pointHash = Double.hashCode(points[i].getX()) ^ Double.hashCode(points[i].getY());
        hash ^= pointHash;
    }

    return hash;
}

```

Возвращает значение хэш-кода для объекта табулированной функции

```

@Override
public Object clone() {
    try {
        FunctionPoint[] clonedPoints = new FunctionPoint[points.length];
        for (int i = 0; i < pointsCount; i++) {
            clonedPoints[i] = (FunctionPoint) points[i].clone();
        }

        ArrayTabulatedFunction cloned = (ArrayTabulatedFunction) super.clone();
        cloned.points = clonedPoints;
        cloned.pointsCount = this.pointsCount;

        return cloned;
    } catch (CloneNotSupportedException e) {
        throw new RuntimeException("клонирование не сработало", e);
    }
}

```

Возвращает объект-копию для объекта табулированной функции

Задание 3

В классе **LinkedListTabulatedFunction** переопределены методы:

```

@Override
public String toString() {
    StringBuilder sb = new StringBuilder("{}");
    FunctionNode current = head.next;
    while (current != head) {
        sb.append(current.point.toString());
        current = current.next;
        if (current != head) {
            sb.append(", ");
        }
    }
    sb.append("{}");
    return sb.toString();
}

```

Возвращает описание табулированной функции двусвязного кольцевого списка

```

@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (!(obj instanceof TabulatedFunction)) return false;

    TabulatedFunction otherFunc = (TabulatedFunction) obj;
    if (this.pointsCount != otherFunc.getPointsCount()) return false;

    if (obj instanceof LinkedListTabulatedFunction) {
        LinkedListTabulatedFunction other = (LinkedListTabulatedFunction) obj;

        FunctionNode thisNode = this.head.next;
        FunctionNode otherNode = other.head.next;

        while (thisNode != head && otherNode != other.head) {
            if (!thisNode.point.equals(otherNode.point)) {
                return false;
            }
            thisNode = thisNode.next;
            otherNode = otherNode.next;
        }
        return true;
    } else {
        for (int i = 0; i < pointsCount; i++) {
            FunctionPoint thisPoint = this.getPoint(i);
            FunctionPoint otherPoint = otherFunc.getPoint(i);
            if (!thisPoint.equals(otherPoint)) {
                return false;
            }
        }
        return true;
    }
}

```

Сравнивает две табулированные функции двусвязного кольцевого списка

```
@Override
public int hashCode() {
    int hash = pointsCount;

    FunctionNode current = head.next;
    while (current != head) {
        hash ^= current.point.hashCode();
        current = current.next;
    }

    return hash;
}
```

Возвращает значение хэш-кода для объекта табулированной функции двусвязного кольцевого списка

```
@Override
public Object clone() {
    FunctionPoint[] pointsArray = new FunctionPoint[pointsCount];
    FunctionNode current = head.next;
    int i = 0;

    while (current != head) {
        pointsArray[i] = (FunctionPoint) current.point.clone();
        current = current.next;
        i++;
    }

    return new LinkedListTabulatedFunction(pointsArray);
}
```

Возвращает объект-копию для объекта табулированной функции двусвязного кольцевого списка

Задание 4

В интерфейс **TabulatedFunction** внесен метод:

```
Object clone();
```

Задание 5

```
=== 1. Тестирование метода toString() ===
```

```
ArrayTabulatedFunction: {(0.0; 0.0), (1.0; 1.0), (2.0; 128.0)}
```

```
LinkedListTabulatedFunction: {(0.0; 0.0), (1.0; 1.0), (2.0; 128.0)}
```

```
=== 2. Тестирование метода equals() ===
```

```
2.1 Сравнение одинаковых ArrayTabulatedFunction:
```

```
arrayFunc1.equals(arrayFunc2) = true
```

```
arrayFunc2.equals(arrayFunc1) = true
```

```
2.2 Сравнение одинаковых LinkedListTabulatedFunction:
```

```
listFunc1.equals(listFunc2) = true
```

```
listFunc2.equals(listFunc1) = true
```

```
2.3 Сравнение ArrayTabulatedFunction и LinkedListTabulatedFunction с одинаковыми точками:
```

```
arrayFunc1.equals(listFunc1) = true
```

```
listFunc1.equals(arrayFunc1) = true
```

```
2.4 Сравнение разных функций:
```

```
arrayFunc1.equals(arrayFunc3) = false
```

```
listFunc1.equals(listFunc3) = false
```

```
arrayFunc1.equals(listFunc3) = false
```

```
=== 3. Тестирование метода hashCode() ===
```

```
3.1 Хэш-коды одинаковых функций:
```

```
hashCode arrayFunc1 = 6291459
```

```
hashCode arrayFunc2 = 6291459
```

```
hashCode listFunc1 = 6291459
```

```
hashCode listFunc2 = 6291459
```

```
3.2 Хэш-коды разных функций:
```

```
hashCode arrayFunc3 = -2144403456
```

```
hashCode listFunc3 = -2144403456
```

```
3.3 Проверка согласованности equals() и hashCode():
```

```
arrayFunc1.equals(arrayFunc2) = true, hashCode равны? true
```

```
listFunc1.equals(listFunc2) = true, hashCode равны? true
```

```
arrayFunc1.equals(arrayFunc3) = false, hashCode равны? false
```

```
3.4 Изменение объекта и хэш-кода:
```

```
Исходный hashCode: 6291459
```

```
hashCode после изменения Y на 0.001: -1827358607
```

```
Объект изменился? true
```

=== 4. Тестирование метода clone() ===

4.1 Клонирование ArrayTabulatedFunction:

Оригинал: {(0.0; 0.0), (1.0; 1.0), (2.0; 128.0)}

Клон: {(0.0; 0.0), (1.0; 1.0), (2.0; 128.0)}

Оригинал == Клон? false

Оригинал.equals(Клон)? true

Оригинал после изменения: {(0.0; 0.0), (1.0; 999.0), (2.0; 128.0)}

Клон после изменения оригинала: {(0.0; 0.0), (1.0; 1.0), (2.0; 128.0)}

Клон не изменился? true

4.2 Клонирование LinkedListTabulatedFunction:

Оригинал: {(0.0; 0.0), (1.0; 1.0), (2.0; 128.0)}

Клон: {(0.0; 0.0), (1.0; 1.0), (2.0; 128.0)}

Оригинал == Клон? false

Оригинал.equals(Клон)? true

Оригинал после изменения: {(0.0; 0.0), (1.0; 888.0), (2.0; 128.0)}

Клон после изменения оригинала: {(0.0; 0.0), (1.0; 1.0), (2.0; 128.0)}

Клон не изменился? true

=== Тестирование завершено ===